

University of Wah

Department of Computer Science



Submit to:

Mr. Hassan Shafaqat

Submit by:

Muhammad Talha

Course:

Artificial Intelligence Lab 201L

Reg No:

UW-24-CS-BS-101

Section:

BS-CS-3rd-B

Final Lab Sessional

Task # 01

(a)

```
print("-----")
print("Muhammad Talh")
print("Muhammad Kamran")
print("22-12-2004")
print("-----")
```

Output

```
-----
Muhammad Talh
Muhammad Kamran
22-12-2004
-----
```

(b)

```
number=20
if number>0:
    print("Number is Positive ")
elif number<0:
    print(" Number is Negative")
else:
    print("Number is Zero")
```

Output

```
Number is Positive
```

(c)

```
string=input()
count=0
for char in string:
    if char.lower() in "aeiou":
        count+=1
print("Vowel Count: ",count)
```

Output

```
Talha
Vowel Count:  2
```

(d)

```
Uni_Name = "UOW"  
Campus = "Wah Cantt"  
Class = "CS-3b"  
Registration_Number = "UW-24-CS-BS-112"  
  
print("University Name ",Uni_Name )  
print("Campus/Location", Campus)  
print("Class/Semester" ,Class)  
print("Registration Number", Registration_Number)
```

Output

```
University Name UOW  
Campus/Location Wah Cantt  
Class/Semester CS-3b  
Registration Number UW-24-CS-BS-112
```

(E)

```
for number in range(1,21):  
    if number%2==0:  
        print(number)
```

Output

```
2  
4  
6  
8  
10  
12  
14  
16  
18  
20
```

Task # 02

```
from collections import deque

graph = {
    'R': ['S', 'T'],
    'S': ['U', 'V'],
    'T': ['W'],
    'U': ['X'],
    'V': ['Y'],
    'W': ['X'],
    'X': ['Y'],
    'Y': []
}

def bfs(start_node):
    visited = set()
    queue = deque([start_node])
    print("BFS Traversal")
    while queue:
        current = queue.popleft()
        if current not in visited:
            print(current, end=" ")
            visited.add(current)
            for neighbor in graph[current]:
                if neighbor not in visited:
                    queue.append(neighbor)
bfs('R')
print()
```

Output

```
BFS Traversal
R S T U V W X Y
```

Task # 03

```
N = 4

def is_safe(board, row, col):
    for i in range(col):
        if board[row][i] == 1:
            return False

    i = row
    j = col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    i = row
    j = col
    while i < N and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1

    return True

def print_board(board):
    for row in board:
        for cell in row:
            if cell == 1:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()

def solve_n_queens(board, col):
    if col >= N:
        return True

    for row in range(N):
        if is_safe(board, row, col):
            board[row][col] = 1

            if solve_n_queens(board, col + 1):
                return True

            board[row][col] = 0

    return False
```

```
board = [[0 for _ in range(N)] for _ in range(N)]

if solve_n_queens(board, 0):
    print("One valid solution for 4-Queens problem:")
    print_board(board)
else:
    print("No solution exists")
```

Output

```
copy
One valid solution for 4-Queens problem:
. . Q .
Q . . .
. . . Q
. Q . .
```